VIBE-AUDIT

Audit de Cybersécurité Express · Format Écran Paysage

DÉMO · application et données fictives

CIBLE	DATE D'AUDIT	AUDITEUR
leadflow-demo.example	13 Octobre 2026	Kreesten Agboton

VULNÉRABILITÉS CONFIRMÉES (4 CONSTATS RÉELS) — 2 ÉLEVÉES · 1 MOYENNE · 1 FAIBLE

2 Élevées (50%) | 1 Moyenne (25%) | 1 Faible (25%)

CONTRÔLES SANS ANOMALIE
4 vérifiés conformes

SYNTHÈSE DÉCIDEUR

L'audit technique passif a identifié 4 vulnérabilités concrètes, principalement liées à l'absence de restrictions Row Level Security dans Supabase et à l'exposition d'une clé API privée côté client. Les infrastructures critiques d'authentification native Supabase sont conformes. L'ensemble des failles peut être résolu en moins d'une heure via les prompts d'assistance IA fournis.

COUVERTURE DE SURFACE & CONFORMITÉ

Composant	Statut	Précision technique
Supabase Rest API & RLS	RLS manquant	Exposition des données transactionnelles
Bundles JS & Secrets	Clé privée exposée	Clé secrète Resend dans le bundle public
En-têtes HTTP & CORS	Headers absents	CSP / X-Frame-Options manquants, CORS *
Supabase Auth & Sessions	Sécurisé	Flux JWT géré nativement avec tokens éphémères
Table 'users' (Isolation)	Sécurisé	Politique RLS active et étanche sur auth.uid()
Chiffrement en transit	Sécurisé	Certificat TLS valide et redirection HTTPS forcée
Passerelle Stripe Checkout	Hors périmètre	Non évalué dans cette mission (infrastructure tierce)

ACTIONS PRIORITAIRES

1. **Activer la Row Level Security sur `orders` (SEC-01)**
   Bloque l'accès public direct aux montants et emails des commandes dans Supabase.

2. **Isoler la clé secrète Resend dans une Edge Function (SEC-02)**
   Élimine le risque d'usurpation d'identité sur vos envois d'emails transactionnels.

3. **Durcir les en-têtes CSP et X-Frame-Options (SEC-03)**
   Protège l'interface cliente contre l'intégration en iframe et le détournement de clics.

Remédiation : Les pages suivantes détaillent chaque preuve technique, le prompt IA testé et les critères de validation.

ÉLEVÉ | Constat confirmé

SEC-01 : Absence de Row Level Security (RLS) sur la table des commandes

supabase/schema:orders | API REST, accès anonyme

1. IMPACT MÉTIER

Fuite de données transactionnelles clients

La table orders est lisible sans filtrage par utilisateur. Tout visiteur muni de la clé publique anon peut extraire l'historique complet des commandes, adresses emails et montants encaissés.

2. PREUVE TECHNIQUE REPRODUCTIBLE

CAPTURE : 13/10/2026 15:30 UTC — GET REST API · 200 OK

```
# Commande passive d'inspection :

curl -i -s "https://leadflow-demo.example/rest/v1/orders?
select=id,customer_email,amount&limit=2" \
  -H "apikey: eyJhbGciOiJIUzI1NiIsInR5cCI**"



# En-tête HTTP certifiant le nombre total d'enregistrements exposés :

Content-Range: 0-1/142

[
  { "id": "ord_812", "customer_email": "m.***@gmail.com", "amount":
4900 },
  { "id": "ord_813", "customer_email": "agence.***@pro.fr", "amount":
12900 }
]



# Constat formel : 142 lignes accessibles publiquement sans session
active
```

3. REMÉDIATION IMMÉDIATE

Correction proposée

CIBLE : LOVABLE / BOLT / CURSOR / SUPABASE SQL — PROMPT PRÊT À COLLER

```
-- Hypothèse : la table 'orders' possède une colonne 'user_id' liée à
auth.users.id



Dans Supabase, active la politique Row Level Security (RLS) sur la table
'orders' :

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Lecture réservée au propriétaire"
ON orders FOR SELECT USING (auth.uid() = user_id);

CREATE POLICY "Insertion réservée au propriétaire"
ON orders FOR INSERT WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Modification réservée au propriétaire"
ON orders FOR UPDATE USING (auth.uid() = user_id);
```

4. Critères de Validation Post-Déploiement

- ○ Test 1 (Après correction) : Rejouer la commande curl avec la clé anon. Le serveur doit renvoyer un tableau vide [] ou un code HTTP 401/403.
- ○ Test 2 (Session utilisateur) : Se connecter avec un compte client : vérifier que ses commandes personnelles restent visibles sans anomalie.
- ○ Re-contrôle J+14 : Prestation éligible au protocole de vérification gratuite après mise

ÉLEVÉ | Constat confirmé

SEC-02 : Clé secrète Resend exposée dans le bundle front-end client

src/config.ts · VITE_RESEND_API_KEY

Inspection statique du bundle JavaScript

1. IMPACT MÉTIER

Usurpation d'identité et envoi d'emails non sollicités

La variable d'environnement RESEND_API_KEY contenant un token de production (re_live_...) est directement accessible dans les sources livrées au navigateur de tout internaute.

2. PREUVE TECHNIQUE REPRODUCTIBLE

```
CAPTURE : 13/10/2026 15:30 UTC                    ANALYSE STATIQUE BUNDLE

# Commande passive d'inspection :

grep -rn "re_live_" dist/assets/*.js


# Clé privée de production détectée en clair dans le code client :

dist/assets/index-C1.js:42

export const RESEND_KEY = "re_live_9a8b***3a2b1c";


# Constat formel : Privilège d'envoi d'emails transactionnels exposé à
tout visiteur
```

3. REMÉDIATION IMMÉDIATE

Correction proposée

```
CIBLE : LOVABLE / BOLT / CURSOR (REFACTORISATION BACKEND) PROMPT PRÊT À COLLER

-- Hypothèse : création d'une Supabase Edge Function dédiée 'send-email'


Supprime VITE_RESEND_API_KEY du fichier .env client et du composant front-end.
Déplace l'envoi d'email dans une Supabase Edge Function sécurisée :

supabase secrets set RESEND_API_KEY=re_live_***

Dans le frontend, remplace l'appel direct par :
await supabase.functions.invoke('send-welcome-email', { body: { userId } });
```

4. Critères de Validation Post-Déploiement

- ○ Test 1 : Regénérer le build front-end (npm run build) et vérifier l'absence totale de la chaîne 're_live_' dans le dossier dist/.
- ○ Test 2 : Révoquer l'ancienne clé dans le dashboard Resend et générer un nouveau secret assigné exclusivement aux Edge Functions.
- ○ Re-contrôle J+14 : Prestation éligible au re-contrôle gratuit.

MOYEN | Constat confirmé

SEC-03 : Absence des en-têtes de sécurité HTTP (CSP, X-Frame-Options)

HTTP Response Headers · leadflow-demo.example

Requête HTTP HEAD passive

1. IMPACT MÉTIER

Vulnérabilité au détournement de clic (Clickjacking)

Sans en-tête X-Frame-Options ni Content-Security-Policy (frame-ancestors), l'application peut être intégrée dans une balise iframe sur un domaine malveillant pour tromper les utilisateurs.

2. PREUVE TECHNIQUE REPRODUCTIBLE

```
CAPTURE : 13/10/2026 15:30 UTC                    HEAD HTTP/1.1 · 200 OK

# Commande passive d'inspection :

curl -I -s "https://leadflow-demo.example"



# En-têtes de sécurité reçus en réponse :

Analyse 4 en-têtes majeurs

Content-Type: text/html
Access-Control-Allow-Origin: *
# Manquant : Content-Security-Policy
# Manquant : X-Frame-Options: DENY
# Manquant : X-Content-Type-Options: nosniff



# Constat formel : Aucune restriction d'encadrement iframe active
```

3. REMÉDIATION IMMÉDIATE

Correction proposée

```
CIBLE : VITE / NEXT.JS / CLOUDFLARE HEADERS                PROMPT PRÊT À COLLER

-- Hypothèse : configuration des en-têtes via le serveur d'hébergement ou le routeur



Ajoute les en-têtes de sécurité HTTP suivants dans la configuration de déploiement (ou headers de Vite/Next.js) :

X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Content-Security-Policy: frame-ancestors 'none';
```

4. Critères de Validation Post-Déploiement

- ○ Test 1 : Rejouer curl -I et constater la présence des en-têtes X-Frame-Options: DENY et CSP.
- ○ Test 2 : Tenter d'ouvrir l'application dans une iframe locale de test : vérifier le blocage par le navigateur.
- ○ Re-contrôle J+14 : Prestation éligible au re-contrôle gratuit.

Accompagnement Inclus

Feuille de Route de Correction Chronologique

`Re-contrôle à J+14 inclus sans frais`

ÉTAPE 1 (IMMÉDIAT - 15 MIN)

Activer RLS sur les tables Supabase

Appliquer le script SQL SEC-01 dans l'éditeur Supabase pour verrouiller instantanément l'accès aux commandes.

ÉTAPE 2 (SOUS 48H - 30 MIN)

Déplacer la clé Resend et révoquer le secret

Transférer l'envoi d'email dans une Edge Function, puis renouveler la clé Resend compromise.

ÉTAPE 3 (MOYEN TERME - 15 MIN)

Ajouter les en-têtes HTTP de sécurité

Configurer les règles CSP et anti-clickjacking dans les fichiers de configuration du reverse proxy ou de l'hébergeur.

PROTOCOLE DE RE-CONTRÔLE GRATUIT (J+14)

Votre audit inclut un **re-contrôle passif complet sous 14 jours calendaires** après la livraison initiale. Pour en bénéficier :

1. Appliquez les prompts de remédiation fournis dans votre interface de développement.
2. Déployez la version corrigée sur votre environnement de production ou staging.
3. Répondez simplement à l'email de remise d'audit en indiquant le commit ou l'URL de test.

Délai de traitement : Nouveau diagnostic passif sous 24h ouvrées et délivrance de l'attestation actualisée.

DÉCHARGE & CONFIDENTIALITÉ (NDA)

- **Méthode strictement passive :** Aucun test de déni de service (DoS) ni altération de données en production.
- **Traitement 100% local :** Aucun code source soumis à des APIs d'entraînement ou services cloud publics.
- **Purge sous 30 jours :** Destruction certifiée et définitive des copies locales et archives de travail.
- **Référence d'accord :** Protocole bilatéral `VA-CONSENT-20261013-DEMO` enregistré avant intervention.